

PRD: Saral Sahayak — AI Agent for Decoding Government Claim Rejections

Track: INFERENTIA — Making Access Easier (secondary fit: Building Smarter Agents)

Team: [Add names]

Date: August 25, 2026

Status: Hackathon build — v1 (24h scope)

1. Problem Statement

When an Indian citizen's EPFO (Provident Fund) claim, Ayushman Bharat (PM-JAY) health claim, or similar government scheme application gets rejected, they receive a **cryptic, jargon-heavy remark** — not an explanation.

- EPFO portal shows short remarks like *"Name mismatch"*, *"UAN inactive"*, *"Claim does not satisfy eligibility conditions"* under Track Claim Status, with no guidance on what to do next.
- PM-JAY hospital denials cite formal rejection codes from the Claims Adjudication Manual that patients and even hospital desks struggle to interpret in the moment.
- Most affected users are not fluent in bureaucratic English/Hindi, don't know which form to refile, which document to fix, or that a formal appeal/grievance process (CGRMS, RTI, 14555 helpline) even exists.

Result: People abandon valid claims — money and healthcare access they are legally entitled to goes unclaimed simply because the system doesn't explain itself.

This is a large-scale, well-documented, currently-unsolved problem — not a hypothetical one.

2. Goal / Value Proposition

Build an AI agent that takes a rejection message (EPFO remark, PM-JAY denial letter, or similar) and turns it into:

1. A plain-language explanation of **why** it was rejected

2. The **exact fix** — which form, which document, which portal
3. A **ready-to-use resubmission / appeal draft**, in the user's preferred language

One-line pitch: *"Turn 'claim rejected' into 'here's exactly what to do,' in plain Hindi or English, in under 30 seconds."*

3. Target Users

- Salaried individuals with rejected EPF withdrawal claims (primary — cleanest data, highest volume)
 - Families with denied/stuck Ayushman Bharat hospital claims (secondary — higher stakes, richer escalation path)
 - Anyone semi-literate in bureaucratic English, low trust/familiarity with government portals
-

4. Scope for the Hackathon (v1)

In scope:

- EPFO PF claim rejection flow, fully built end-to-end (primary demo path)
- PM-JAY denial flow, built if time allows (stretch — strong for judges given higher emotional stakes)
- Curated rejection-category knowledge base (from public EPFO/PM-JAY member-facing documentation — no live government API access exists, so this is a structured dataset we build, not scraped/live data)
- Bilingual output (English + Hindi)

Out of scope for v1:

- Live integration with actual EPFO/NHA systems (no public API exists)
 - Any other government schemes beyond EPFO + PM-JAY
 - User accounts / persistence / login
 - Legal guarantee of appeal success (we are a guidance tool, not a legal service — must be stated clearly in UI)
-

5. Core User Flow

1. User pastes/uploads their rejection text (EPFO remark or PM-JAY denial letter/photo)

2. User optionally selects language (English / Hindi)
 3. System processes through agent pipeline (see Section 6) — pipeline steps shown live on screen
 4. Output:
 - Plain-language explanation of the rejection reason
 - Step-by-step fix checklist (which form, which document, who signs/attests)
 - Downloadable, pre-filled resubmission letter or appeal draft
 - (PM-JAY only) Escalation path if unresolved: CGRMS portal link, 14555 helpline, RTI template
-

6. Agent Architecture

Multi-agent pipeline, orchestrated with parallel dispatch where steps are independent.

Agent	Role	Input	Output
Extractor	OCR (if image) + parses raw rejection text into structured fields	Raw text/image	Rejection code/remark, claim ID, dates, amount
Classifier	Maps extracted remark to one of the known rejection categories in our curated dataset	Structured fields	Category (e.g., “Aadhaar name mismatch”, “eligibility not met”, “bank details error”)
Explainer	Translates category into plain-language explanation, in chosen language	Category	Plain-language explanation
Fix Generator	Looks up the specific remedy for the category — form name, required document, who needs to act	Category	Step-by-step fix checklist
Draft Agent	Generates the actual resubmission form text / appeal letter / RTI request, pre-filled with user’s details	Category + user details	Downloadable draft document

Orchestrator	Coordinates the above, merges into final response	—	Final structured output
---------------------	---	---	-------------------------

Why this counts as a real multi-agent system: Extractor and Classifier can run per-document; Fix Generator and Draft Agent can dispatch in parallel once category is known; each agent has a scoped prompt and doesn't need the others' full context to do its job.

7. Data Sources

Since no live EPFO/NHA rejection-code API is public, v1 relies on a **curated, hand-built dataset** sourced from publicly documented, member-facing rejection patterns:

EPFO categories (confirmed via public guidance):

- Aadhaar / PAN / UAN / bank record name or DOB mismatch
- Inactive or unlinked UAN
- Insufficient PF balance for amount claimed
- Minimum service period not met
- Withdrawal reason not covered under EPF Scheme rules
- Incorrect claim type selected
- Incorrect bank account number / IFSC / joint account used
- Incomplete or mismatched KYC
- Missing Date of Exit (DOE) entry
- PAN not updated (TDS-related rejection for <5 years service)

PM-JAY categories (from Claims Adjudication Manual + citizen guidance):

- Documentation/pre-authorization errors
- Treatment package mismatch or coding issues
- Beneficiary eligibility/verification failure
- Hospital empanelment or fraud-flag related rejection

This dataset is the “ground truth” the Classifier agent matches against — this is what makes the tool grounded rather than hallucinating an explanation.

Escalation resources (PM-JAY): CGRMS portal (cgrms.nha.gov.in), PM-JAY helpline 14555, RTI request template.

8. Tech Stack

- **Backend:** FastAPI (Python), agents as separate LLM calls (Claude API) with distinct system prompts
 - **OCR/input:** Vision-capable LLM call directly on uploaded image (skip separate OCR pipeline to save build time)
 - **Knowledge base:** Static JSON/CSV of rejection categories + fixes, loaded locally (no DB needed for v1)
 - **Frontend:** Next.js or Streamlit — upload/paste box live agent pipeline visualization output panel with downloadable draft
 - **Language handling:** Prompt-level bilingual generation (no separate translation service needed)
-

9. Demo Script (2–3 min)

1. Show a real EPFO rejection remark (prepared sample) — paste it in
 2. Live agent pipeline animates: Extract Classify Explain Fix Draft
 3. Reveal plain-language explanation: *“Your claim was rejected because your name in EPFO records doesn’t exactly match your Aadhaar — this is one of the most common rejection reasons.”*
 4. Show the fix checklist and the pre-filled resubmission draft, downloadable
 5. Switch language toggle to Hindi live — same output, localized
 6. (If built) Switch to PM-JAY flow, show the escalation path — CGRMS + RTI draft — for the higher-stakes healthcare case
 7. Close with the scale of the problem: EPF withdrawal is one of the most common financial interactions for salaried Indians, and rejection explainers are a cottage industry of blog content — evidence this pain is real and unsolved.
-

10. Success Metrics for Judging

- **Working end-to-end demo** on real (sample) rejection text, not scripted/faked output
 - **Grounded, not hallucinated** — explanation traceable to the curated category dataset
 - **Genuine multi-agent architecture** — visible parallel/staged agent execution, not one prompt pretending to be many
 - **Real-world groundedness** — cites actual EPFO/PM-JAY processes, portals, and escalation paths
 - **Accessibility angle** — bilingual output, plain language, addresses a documented equity gap
-

11. Build Plan / Timeline (24h)

Time	Task
0–2h	Finalize rejection-category dataset (EPFO first), set up repo, agent prompt skeletons
2–6h	Build Extractor + Classifier agents, test against sample rejection texts
6–10h	Build Explainer + Fix Generator + Draft Agent, wire up orchestration
10–14h	Frontend: upload flow, pipeline visualization, output panel
14–18h	Bilingual output, polish drafts/downloads, PM-JAY flow if time allows
18–22h	End-to-end testing with multiple sample inputs, fix bugs
22–24h	Demo script rehearsal, slides/README, buffer

12. Open Questions / Risks

- **Risk:** No live EPFO API — dataset must be manually curated and clearly labeled as such (don't overclaim live integration).
- **Risk:** OCR accuracy on photographed rejection letters — mitigate by allowing text paste as primary input, image as bonus.
- **Question:** Do we scope to EPFO only, or attempt both EPFO + PM-JAY?
Recommend: EPFO fully polished first, PM-JAY only if ahead of schedule.
- **Disclaimer needed:** Tool provides guidance, not legal/financial advice — must be stated in UI.